Ejemplos prácticos para cada una de las técnicas de data mining:

1. Descriptivas:

- Ejemplo: Supongamos que tienes una base de datos de ventas en una tienda online. Utilizando técnicas descriptivas, puedes analizar patrones de compra, como identificar los productos más vendidos, los días de la semana con mayor actividad de compras, y las tendencias estacionales. Esto te proporciona una visión general de cómo se comportan tus clientes y te ayuda a tomar decisiones informadas sobre inventario y estrategias de marketing.

2. Predictivas:

- Ejemplo: Imagina que trabajas en una compañía de telecomunicaciones y deseas prever la cancelación de servicios (churn) de tus clientes. Utilizando técnicas predictivas, puedes analizar datos históricos para identificar patrones que indican un mayor riesgo de cancelación. Con esta información, puedes desarrollar un modelo predictivo que clasifique a los clientes en categorías de riesgo y tomar medidas proactivas para retener a aquellos con mayor probabilidad de cancelar.

3. Prescriptivas:

- Ejemplo: Supongamos que eres el gerente de una cadena de restaurantes y deseas optimizar el menú para aumentar las ventas. Utilizando técnicas prescriptivas, puedes recopilar datos sobre las preferencias de los clientes, los costos de los ingredientes y las tendencias del mercado. Un modelo prescriptivo podría recomendarte cambios específicos en el menú, como la introducción de nuevos platos populares o ajustes en los precios, para maximizar los ingresos y la satisfacción del cliente.

Estos ejemplos ilustran cómo las técnicas de data mining pueden utilizarse en diferentes contextos para obtener información valiosa, prever tendencias futuras y tomar decisiones estratégicas basadas en los datos.

LA ACTIVIDAD:

Crearemos una base de datos sencilla para un escenario que pueda ilustrar los ejemplos anteriores. Vamos a crear una base de datos ficticia para una tienda online que vende productos electrónicos. La base de datos tendrá información sobre las ventas, los clientes y los productos.

1. **Descriptivas:**
   - Crearemos una tabla ventas con información sobre las transacciones, como fecha, producto vendido y cantidad.

```
Fecha,Producto,Cantidad
2024-01-01,Laptop,3
2024-01-02,Smartphone,5
2024-01-03,Tablet,2
2024-01-04,Laptop,2
2024-01-05,Smartwatch,1
```

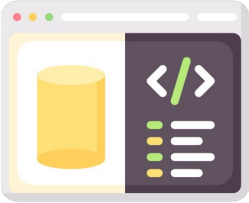
2. **Predictivas:**
   - Crearemos otra tabla clientes que incluirá información sobre los clientes, como su historial de compras.

```
Cliente,Total_Compras,Meses_Activo,Churn
Cliente1,500,12,0
Cliente2,200,6,1
Cliente3,1000,24,0
Cliente4,300,9,1
Cliente5,150,8,0
```

3. **Prescriptivas:**
   - Por último, crearemos una tabla productos que contendrá información sobre los productos, incluyendo sus costos y popularidad.

```
Producto,Costo,Popularidad
Laptop,800,4
Smartphone,500,5
Tablet,300,3
Smartwatch,200,2
```

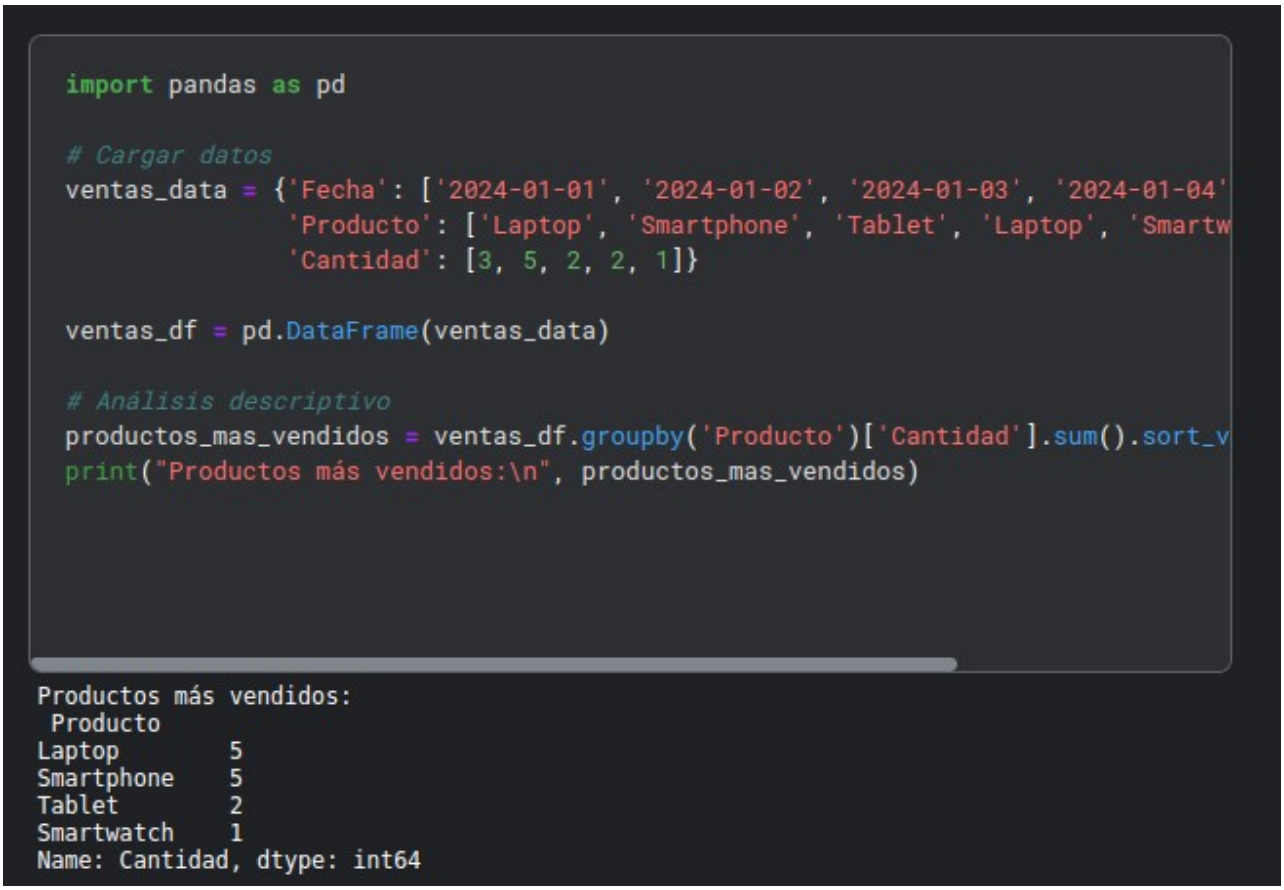
Estos son ejemplos en formato CSV. Para un entorno de Kaggle, puedes cargar estos datos ficticios como archivos CSV y luego ejecutar código Python para realizar análisis descriptivos, predictivos y prescriptivos según los ejemplos anteriores.



Código para cada caso utilizando Python y bibliotecas comunes como **pandas y scikit-learn:**

1. Descriptivas:

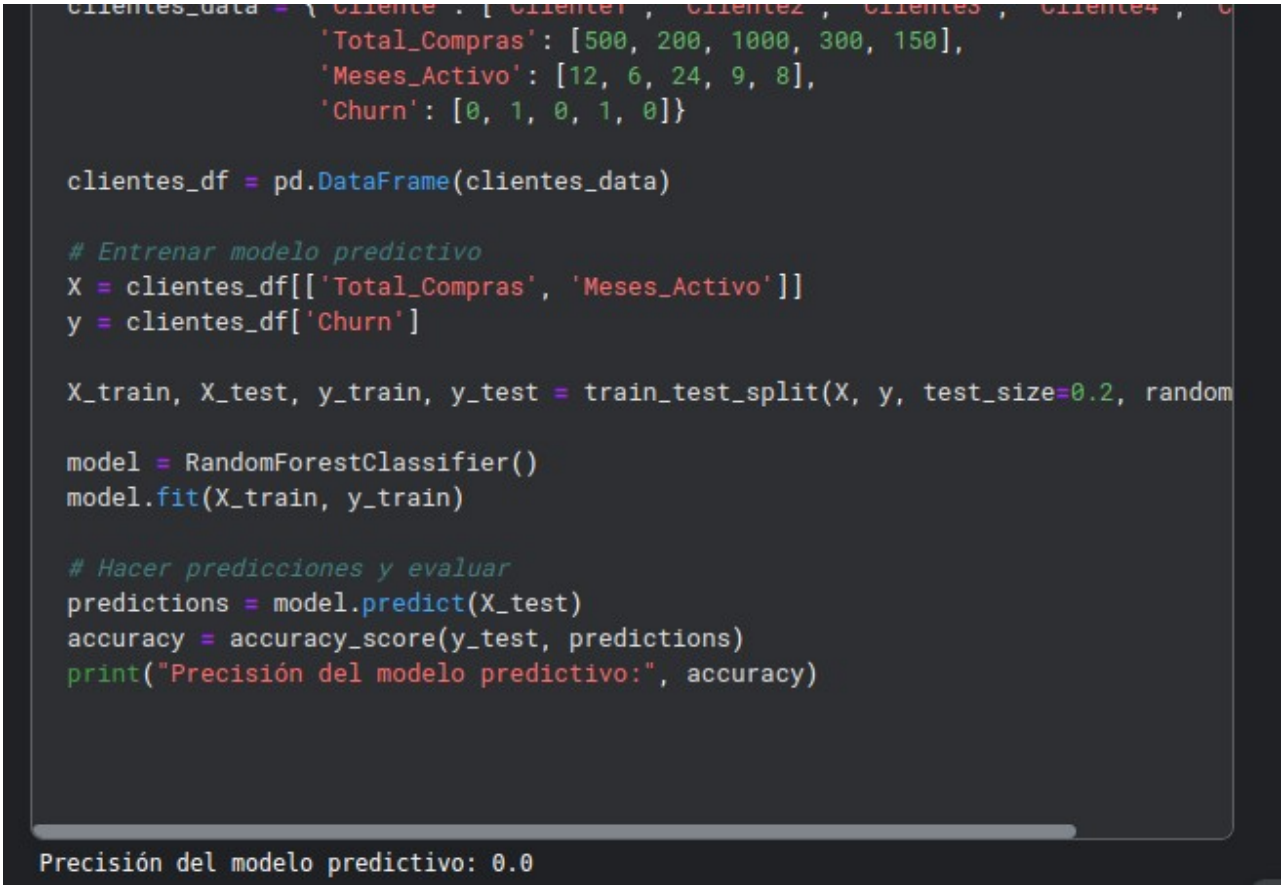
```
import pandas as pd
# Cargar datos
ventas_data = {'Fecha': ['2024-01-01', '2024-01-02', '2024-01-03',
'2024-01-04', '2024-01-05'],
               'Producto': ['Laptop', 'Smartphone', 'Tablet',
'Laptop', 'Smartwatch'],
               'Cantidad': [3, 5, 2, 2, 1]}
ventas_df = pd.DataFrame(ventas_data)
# Análisis descriptivo
productos_mas_vendidos = ventas_df.groupby('Producto')
['Cantidad'].sum().sort_values(ascending=False)

print("Productos más vendidos:\n", productos_mas_vendidos)
```



La salida no saca gráficos sino datos estadísticos, matemáticos que nos aportan información relevante.

2. Predictivas:

```
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score
# Cargar datos
clientes_data = {'Cliente': ['Cliente1', 'Cliente2', 'Cliente3',
'Cliente4', 'Cliente5'],
                 'Total_Compras': [500, 200, 1000, 300, 150],
                 'Meses_Activo': [12, 6, 24, 9, 8],
                 'Churn': [0, 1, 0, 1, 0]}
clientes_df = pd.DataFrame(clientes_data)
# Entrenar modelo predictivo
X = clientes_df[['Total_Compras', 'Meses_Activo']]
y = clientes_df['Churn']
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, random_state=42)
model = RandomForestClassifier()
model.fit(X_train, y_train)
# Hacer predicciones y evaluar
predictions = model.predict(X_test)
accuracy = accuracy_score(y_test, predictions)
print("Precisión del modelo predictivo:", accuracy)
```



En esta caso la salida muestra la evaluación del algoritmo para éste modelo predictivo.

3. Prescriptivas:

```
# Cargar datos
productos_data = {'Producto': ['Laptop', 'Smartphone', 'Tablet',
'Smartwatch'],
                  'Costo': [800, 500, 300, 200],
                  'Popularidad': [4, 5, 3, 2]}
productos_df = pd.DataFrame(productos_data)
# Recomendación de productos populares y rentables
productos_recomendados = productos_df[(productos_df['Popularidad']
> 3) & (productos_df['Costo'] < 600)]
print("Productos recomendados:\n", productos_recomendados)
```

Estos códigos son muy básicos y podrían ser mejorados y adaptados según la complejidad de tus datos y tus objetivos específicos en Kaggle.

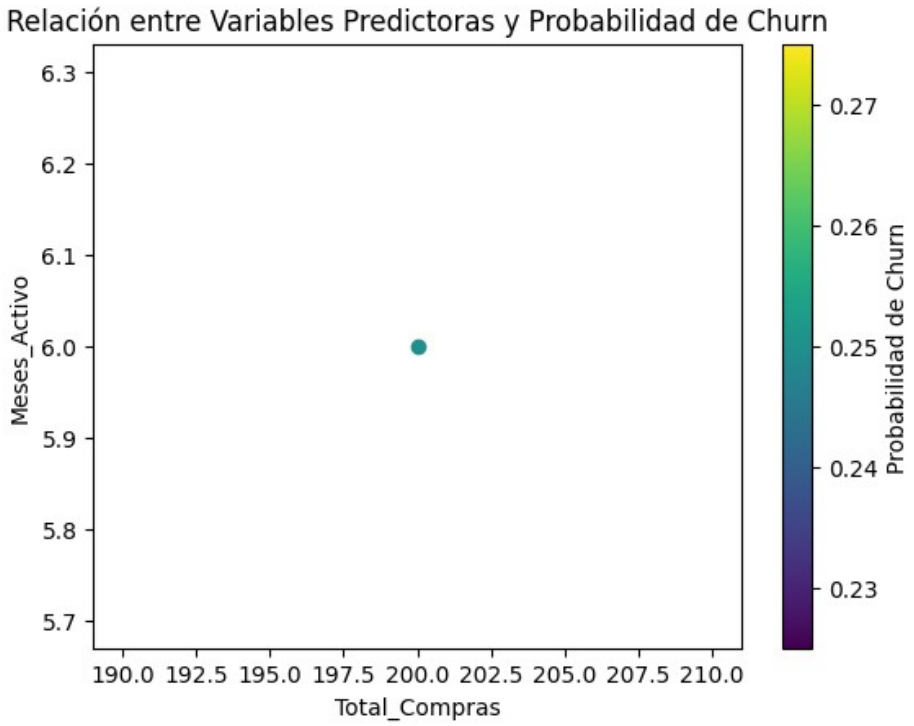
En esta salida crea una recomendación basada en cruce de los datos que aporta el conocimiento para tomar una decisión estratégica.

Para crear una **gráfica predictiva**, puedes visualizar la relación entre las variables predictoras y la variable objetivo en el conjunto de datos de prueba. En este caso, usaremos un gráfico de dispersión para mostrar cómo las características Total_Compras y Meses_Activo se relacionan con la probabilidad de Churn (cancelación de servicios) según el modelo predictivo.

Vamos a modificar el código predictivo para incluir la visualización:

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score
# Cargar datos
clientes_data = {'Cliente': ['Cliente1', 'Cliente2', 'Cliente3',
'Cliente4', 'Cliente5'],
                 'Total_Compras': [500, 200, 1000, 300, 150],
                 'Meses_Activo': [12, 6, 24, 9, 8],
                 'Churn': [0, 1, 0, 1, 0]}
clientes_df = pd.DataFrame(clientes_data)
# Entrenar modelo predictivo
X = clientes_df[['Total_Compras', 'Meses_Activo']]
y = clientes_df['Churn']
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, random_state=42)
model = RandomForestClassifier()
model.fit(X_train, y_train)
# Visualización de la relación entre variables predictoras y Churn
plt.scatter(X_test['Total_Compras'], X_test['Meses_Activo'],
c=model.predict_proba(X_test)[:, 1], cmap='viridis', marker='o')
plt.xlabel('Total_Compras')
plt.ylabel('Meses_Activo')
plt.title('Relación entre Variables Predictoras y Probabilidad de
Churn')
plt.colorbar(label='Probabilidad de Churn')
plt.show()
# Hacer predicciones y evaluar
predictions = model.predict(X_test)
accuracy = accuracy_score(y_test, predictions)
print("Precisión del modelo predictivo:", accuracy)
```

En este código, la visualización utiliza un gráfico de dispersión donde el color de cada punto representa la probabilidad predicha de Churn según el modelo. Puedes ajustar la escala de colores y el marcador según preferencias.



Precisión del modelo predictivo: 0.0